

EREDITARIETÀ E DYNAMIC DISPATCH

Ereditarietà per estensione

In Java è possibile definire una classe come estensione di un'altra classe esistente, usando la parola chiave **extends**

▶ `class B extends A {...}`

In questo caso, A è **super-classe**, mentre B è **sotto-classe**

Osservazione:

Java si basa su **ereditarietà singola** (o **semplice**)

- ▶ Una classe può implementare **più interfacce**, ma può estendere **una sola super-classe**
- ▶ Altri linguaggi (ad es. C++) prevedono ereditarietà multipla

La classe Object

Una classe che non estende altre classi, implicitamente estende la classe di default **Object**

▶ `class A {...} // implicitamente estende Object`

La classe Object fornisce alcuni **metodi** che sono **disponibili per tutte le classi**

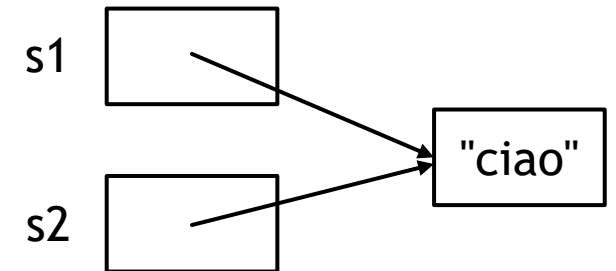
- ▶ `toString()` - restituisce una rappresentazione testuale dell'oggetto
- ▶ `equals(obj)` - confronta l'oggetto corrente con l'oggetto `obj`
- ▶ `clone()` - crea una copia dell'oggetto
- ▶ ... vedere <https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html>

Nota importante: confronto con equals()

Tra i metodi di Object, è importante sottolineare `equals`

- ▶ Consente di **confrontare** due oggetti dal punto di vista dei **contenuti**!
- ▶ Ogni classe può ridefinire il metodo `equals` in modo da realizzare un confronto sensato per il tipo di dato rappresentato dalla classe
- ▶ **Attenzione**: confrontare con `==` corrisponde invece a valutare se i riferimenti a quegli oggetti sono uguali (cioè sono **alias**)

```
String s1 = "ciao"; // modo per scrivere un oggetto
String s2 = s1;
String s3 = "ciao a tutti";
String s4 = s3.substring(0,4); // restituisce sottostringa "ciao"
System.out.println(s1==s2);           // true (stesso riferimento)
System.out.println(s2==s4);           // false (riferimenti diversi)
System.out.println(s2.equals(s4));    // true (stesso contenuto)
System.out.println(s2.equals(s3));    // false (contenuti diversi)
```



Nota importante: confronto con equals()

Attenzione String è speciale...

- I valori letterali ("abcd") sono trattati come degli oggetti, ma sono IMMUTABILI
- I metodi di String non modificano gli oggetti, ma ne restituiscono sempre dei nuovi come risultato
- Il compilatore riconosce valori letterali uguali nel codice e ottimizza il codice costruendo l'oggetto corrispondente una volta sola:

```
String s1 = "ciao";
```

```
String s2 = "ciao";
```

```
System.out.println(s1 == s2); // true (per l'ottimizzazione)
```

```
System.out.println(s2==s4); // false (riferimenti diversi)
```

```
System.out.println(s2.equals(s4)); // true (stesso contenuto)
```

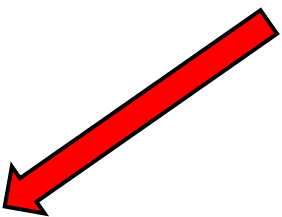
Modificatori di visibilità e sotto-classi

Le **regole di visibilità** legate ai modificatori public e private valgono anche tra **super-classe** e **sotto-classe**

```
class A {  
    private int a1;  
    private int a2;  
    ...  
}
```

**B non può accedere ad a1 e a2
perché sono membri privati di A**

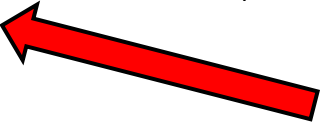
```
class B extends A {  
    private int a3;  
    public int somma() {  
        return a1 + a2 + a3;  
    }  
}
```



Modificatori di visibilità e sotto-classi

Le **regole di visibilità** legate ai modificatori `public` e `private` valgono anche tra **super-classe** e **sotto-classe**

```
class A {  
    protected int a1;  
    protected int a2;  
    ...  
}
```



Il modificatore **protected** rende i
membri **visibili** alle sotto-classi **senza**
renderli completamente **pubblici**

```
class B extends A  
    private int a3;  
    public int somma() {  
        return a1 + a2 + a3;  
    }  
}
```

Altrimenti B dovrebbe accedere ad `a1` e `a2` tramite opportuni **metodi pubblici** di A

- è un'altra soluzione che può aver senso...

Classi astratte

In una gerarchia di classi, può aver senso avere delle **classi parzialmente definite** (**classi astratte**) per condividere comportamenti comuni tra più classi

Una **classe astratta** è una classe che **contiene almeno un metodo astratto**, ossia presente nella classe, ma **non implementato** (come accade nelle interfacce): ad es. `public abstract void absmethod()` ;

- ▶ Una classe astratta **non può essere istanziata** direttamente
- ▶ Una classe astratta **può essere estesa** da una classe che si occupa di implementarne i metodi astratti

Ha lo **scopo** di **definire un tipo di oggetto e di metodi** comuni a più classi (come le interfacce), fornendo anche **l'implementazione di alcuni metodi comuni** (a differenza delle interfacce)

Classi astratte

```
public abstract class Solido {  
    // variabile d'istanza  
    private double pesoSpecifico ;  
  
    // costruttore  
    public Solido ( double ps ){  
        pesoSpecifico = ps;  
    }  
  
    // metodo implementato  
    public double peso (){  
        return volume () * pesoSpecifico ;  
    }  
  
    // metodi astratti  
    public abstract double volume ();  
    public abstract double superficie ();  
}
```

Classe astratta che rappresenta un **solido generico**, base per creare altre classi che rappresentano tipi specifici di solidi, fornendo un'implementazione comune e lasciando alcune parti (i metodi astratti) da definire nelle sottoclassi

Classi astratte

```
public abstract class Solido {  
    // variabile d'istanza  
    private double pesoSpecifico ;  
  
    // costruttore  
    public Solido ( double ps ){  
        pesoSpecifico = ps;  
    }  
  
    // metodo implementato  
    public double peso () {  
        return volume () * pesoSpecifico ;  
    }  
  
    // metodi astratti  
    public abstract double volume () ;  
    public abstract double superficie () ;  
}
```

Classe astratta che rappresenta un **solido generico**, base per creare altre classi che rappresentano tipi specifici di solidi, fornendo un'implementazione comune e lasciando alcune parti (i metodi astratti) da definire nelle sottoclassi

Le classi concrete che estendono Solido devono fornire un'implementazione specifica dei metodi astratti che saranno specifici del solido scelto

Classi astratte

Classe completa: la si può istanziare

```
public abstract class Solido {  
    // variabile d'istanza  
    private double pesoSpecifico ;  
  
    // costruttore  
    public Solido ( double ps ){  
        pesoSpecifico = ps;  
    }  
  
    // metodo implementato  
    public double peso (){  
        return volume () * pesoSpecifico  
    }  
  
    // metodi astratti  
    public abstract double volume ();  
    public abstract double superficie ();  
}
```

```
public class Sfera extends Solido {  
    // variabili d'istanza (oltre a pesoSpecifico)  
    private double raggio;  
    // costruttore  
    public Sfera(double raggio, double ps) {  
        super(ps); // chiama il costruttore di Solido  
        this.raggio = raggio; // usa this per disambiguare  
    }  
    // implementazione dei metodi astratti di Solido  
    public double volume() {  
        return 4/3 * Math.PI * Math.pow(raggio, 3);  
    }  
    public double superficie() {  
        return 4 * Math.PI * raggio * raggio ;  
    }  
    // peso() è definito nella superclasse  
}
```

Classi astratte

Classe completa: la si può istanziare

```
public abstract class Solido {  
    // variabile d'istanza  
    private double pesoSpecifico ;  
  
    // costruttore  
    public Solido ( double ps ){  
        pesoSpecifico = ps;  
    }  
  
    // metodo astratto volume() e superficie() rendono la  
    // classe adatta a rappresentare diversi tipi di solidi, sfruttando il  
    // polimorfismo per adattare il comportamento specifico in base  
    // alla sottoclasse del solido specifico  
    public abstract double volume ();  
    public abstract double superficie ();  
}
```

I metodi astratti `volume()` e `superficie()` rendono la classe adatta a rappresentare diversi tipi di solidi, sfruttando il polimorfismo per adattare il comportamento specifico in base alla sottoclasse del solido specifico

```
public class Sfera extends Solido {  
    // variabili d'istanza (oltre a pesoSpecifico)  
    private double raggio;  
    // costruttore  
    public Sfera(double raggio, double ps) {  
        super(ps); // chiama il costruttore di Solido  
        raggio = raggio; // usa this per disambiguare  
    }  
    // implementazione dei metodi astratti di Solido  
    public double volume() {  
        return 4/3 * Math.PI * Math.pow(raggio, 3);  
    }  
    public double superficie() {  
        return 4 * Math.PI * raggio * raggio ;  
    }  
    // peso() è definito nella superclasse  
}
```

Gerarchia di classi e polimorfismo

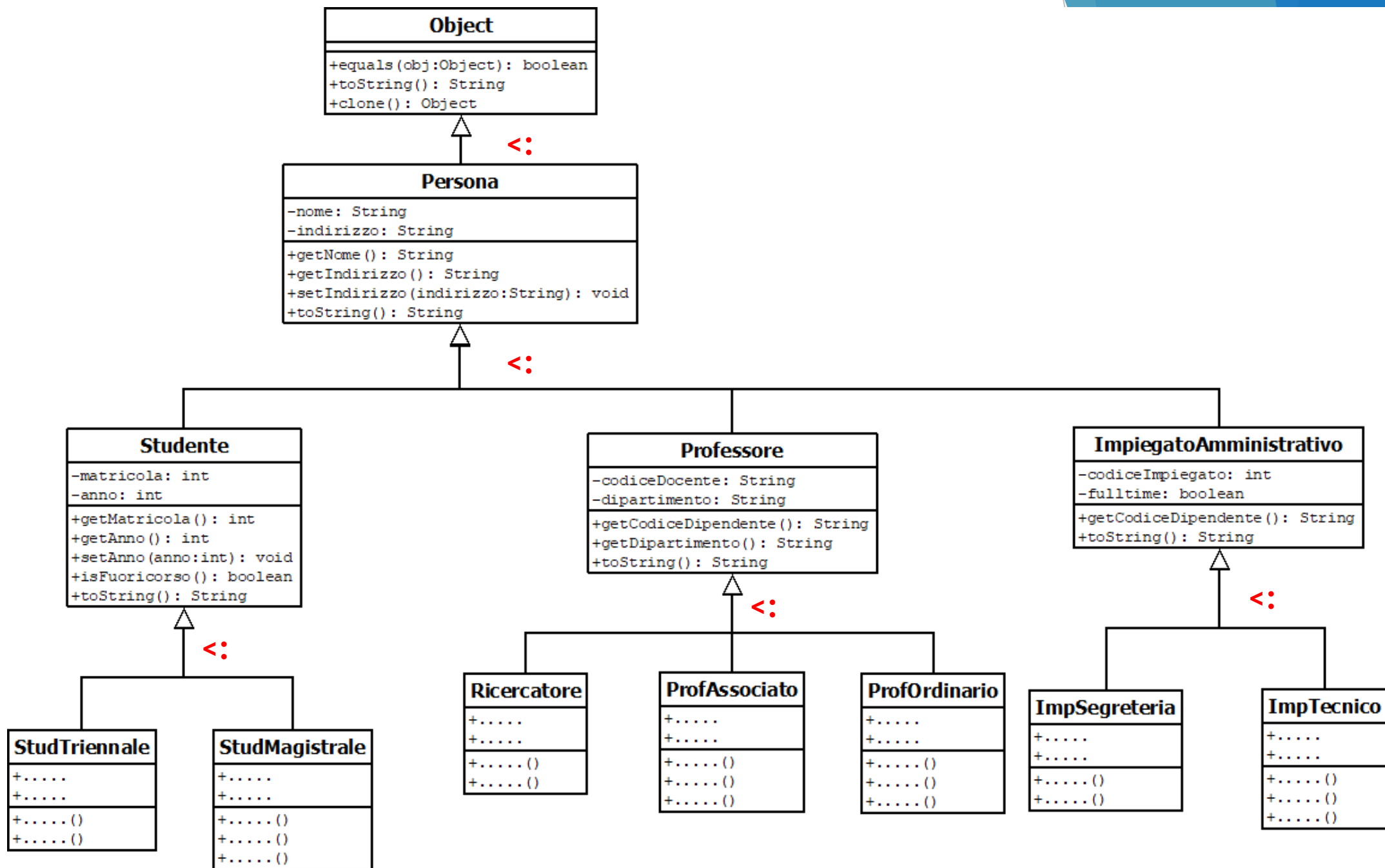
Il costrutto `extends` quindi consente di creare una **gerarchia di classi** organizzata in una struttura ad **albero** la cui radice è `Object`

- ▶ In Java a ogni **classe** (e interfaccia) è associato un **tipo di oggetto**
- ▶ Per il **Nominal Subtyping** la relazione tra le classi in una gerarchia (estesa con anche le interfacce) corrisponde a una **relazione di sottotipo** **<:**, in cui una classe più specifica è considerata un sottotipo di una classe più generale
- ▶ l'elemento **"Top"** è **Object** che, essendo al vertice della gerarchia, è il supertipo di tutte le classi e interfacce

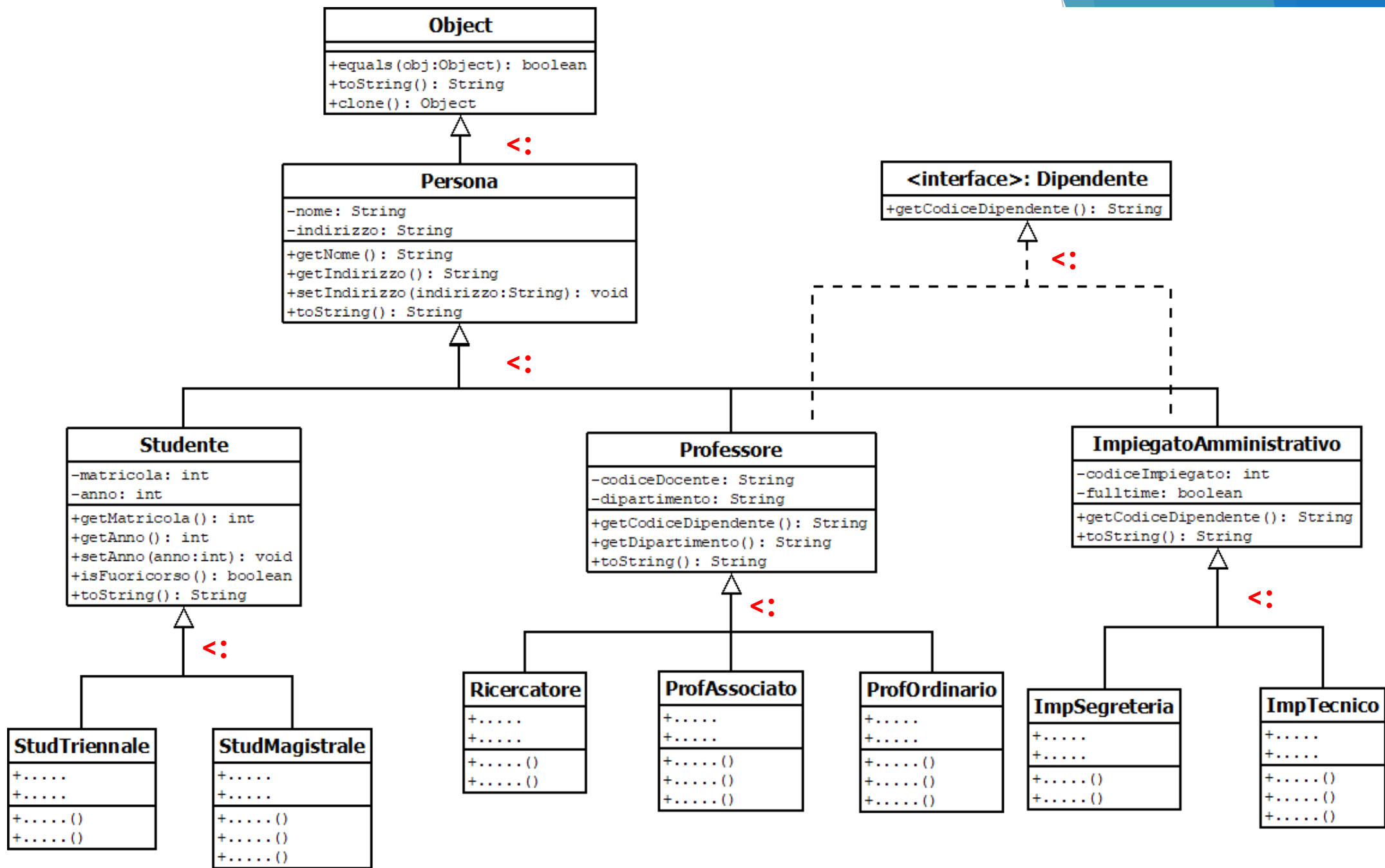
$$\frac{\Gamma \vdash_e \text{exp} : S \quad S <: T}{\Gamma \vdash_e \text{exp} : T}$$

- ▶ La regola di **subsumption** del sistema di tipi, automaticamente, ci consente allora di ottenere un meccanismo di **polimorfismo per sottotipo**

Esempio di gerarchia di classi



Esempio di gerarchia di classi (con interfacce)



```
class Persona {...}

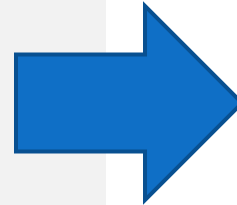
interface Dipendente {...}

class Studente extends Persona {...}
class Professore extends Persona
    implements Dipendente {...}
class ImpAmministrativo extends Persona
    implements Dipendente {...}

class StudTriennale extends Studente {...}
class StudMagistrale extends Studente {...}

class Ricercatore extends Professore {...}
class ProfAssociato extends Professore {...}
class ProfOrdinario extends Professore {...}

class ImpSegreteria extends ImpAmministrativo {...}
class ImpTecnico extends ImpAmministrativo {...}
```



```
Persona <: Object
Dipendente <: Object

Studente <: Persona
Professore <: Persona
Professore <: Dipendente
ImpAmministrativo <: Persona
ImpAmministrativo <: Dipendente

StudTriennale <: Studente
StudMagistrale <: Studente

Ricercatore <: Professore
ProfAssociato <: Professore
ProfOrdinario <: Professore

ImpSegreteria <: ImpAmministrativo
ImpTecnico <: ImpAmministrativo
```



```
class Persona { }
```

```
in
```

Proprietà transitiva $\frac{T_1 <: T_2 \quad T_2 <: T_3}{T_1 <: T_3}$

Subsumption $\frac{\Gamma \vdash_e \text{exp} : S \quad S <: T}{\Gamma \vdash_e \text{exp} : T}$

New $\Gamma \vdash_e \text{new } T() : T$

Assegnamento $\frac{\Gamma \vdash_e \text{exp} : T}{\Gamma \vdash_c T \ x = \text{exp}}$

Con queste regole possiamo derivare:

$\Gamma \vdash_c \text{Persona } p = \text{new StudTriennale}();$

Persona <: Object

Dipendente <: Object

Studiante <: Persona

Professore <: Persona

Professore <: Dipendente

ImpAmministrativo <: Persona

ImpAmministrativo <: Dipendente

StudTriennale <: Studiante

StudMagistrale <: Studiante

Ricercatore <: Professore

ProfAssociato <: Professore

ProfOrdinario <: Professore

ImpSegreteria <: ImpAmministrativo

ImpTecnico <: ImpAmministrativo

Structural vs Nominal Subtyping

Teorema

Sia $<:$ la relazione di tipo inferita dalla gerarchia di classi di Java. Per ogni coppia di classi S e T tale che $S <: T$ (cioè S è un sottotipo di T), ogni membro pubblico di T è automaticamente anche membro pubblico di S

Dimostrazione

Banale... segue dalle definizioni di (i) `extends` (le sotto-classi ereditano tutti i membri pubblici (e protetti) delle super-classi) e di (ii) `implements`, (le classi che implementano un'interfaccia devono fornire (o ereditare) l'implementazione di tutti i metodi pubblici definiti dall'interfaccia), nonché (iii) dalla transitività di $<:$

Conseguenza

Structural subtyping $<:_s$ è una relazione **più debole** del Nominal Subtyping $<:_n$

$$S <:_n T \Rightarrow S <:_s T$$

$$S <:_s T \not\Rightarrow S <:_n T$$

Structural vs Nominal Subtyping

Conseguenza

Structural subtyping $<::_s$ è una relazione più debole del Nominal Subtyping $<::_n$

$$S <::_n T \Rightarrow S <::_s T$$

$$S <::_s T \not\Rightarrow S <::_n T$$

- ▶ $S <::_n T$ (nominal): solo se esplicitamente dichiarato nella gerarchia.
- ▶ La dichiarazione esplicita garantisce che S possieda i membri di T
- ▶ $S <::_s T$ (strutturale): se S ha tutti i membri pubblici richiesti da T , indipendentemente dalla dichiarazione esplicita.
- ▶ S potrebbe quindi avere una struttura compatibile con T (stessi membri pubblici) senza che sia stata dichiarata una relazione esplicita tra le due

Overloading, overriding e
dynamic dispatch

Invocare un metodo...

```
public class Esami {  
    ...  
    public boolean verbalizza(Professore p, Studente s, ...) { ... }  
}  
  
Esami e = new Esami();  
ProfAssociato prof = new ProfAssociato("Mario Rossi",...);  
StudTriennale stud = new StudTriennale("Rosa Bianchi",...);  
e.verbalizza(prof, stud, ...); // chiamata del metodo  
// ProfAssociato <: Professore, StudTriennale <: Studente, OK!
```

Che succede quando **invochiamo un metodo su un oggetto**?

- ▶ Innanzitutto, la chiamata deve superare i **controlli statici** operati dal compilatore
 - ▶ Il **metodo deve essere presente nella classe** che descrive il **tipo apparente** (o **tipo statico**) dell'oggetto, ossia quello della **dichiarazione**
 - ▶ I **parametri attuali** devono essere **compatibili** con il tipo dei **parametri formali** del metodo (sottotipi)

Overloading di metodi

In una stessa classe Java si possono definire più metodi con lo stesso nome (**overloading** di metodi)...

- I metodi in overloading devono però avere **FIRME DIVERSE** (parametri diversi in numero o tipo)

Metodo	Firma (Signature)
int getVal()	getVal()
int minimo(int x, int y)	minimo(int, int)
int minimo(int a, int b)	minimo(int, int)
Double minimo(int a, int b)	minimo(int,int)
double minimo(double x, double y)	minimo(double, double)
int minimo(int x, int y, int z)	minimo(int, int, int)

Quali di questi metodi potrebbero stare nella stessa classe?

Overloading di metodi

Domande...

- Perché due metodi con la stessa firma non possono stare nella stessa classe?
- Perché il nome dei parametri formali e il tipo del risultato del metodo non fanno parte della firma?

Metodo	Firma (Signature)
int getVal()	getVal()
int minimo(int x, int y)	minimo(int, int)
int minimo(int a, int b)	minimo(int, int)
double minimo(int a, int b)	minimo(int,int)
double minimo(double x, double y)	minimo(double, double)
int minimo(int x, int y, int z)	minimo(int, int, int)

Quali di questi metodi potrebbero stare nella stessa classe?

Overloading di metodi

Domande...

- Perché due metodi con la stessa firma non possono stare nella stessa classe?
- Perché il nome dei parametri formali e il tipo del risultato del metodo non fanno parte della firma?

Risposte:

- La firma riassume le **informazioni** che, a run time, possono essere **dedotte dalla chiamata** del metodo
 - Ad es. da `obj.minimo(7, 9)` si può dedurre il nome del metodo, il numero e i tipi dei parametri)e che possono essere **usate per capire quale metodo chiamare**
- Il nome dei parametri formali e il tipo restituito **non possono invece essere dedotti** dalla chiamata. Durante una chiamata al metodo, il compilatore non può distinguere quale metodo invocare solo basandosi sul **tipo di ritorno**, mentre i nomi dei **parametri formali** sono rilevanti per il loro utilizzo interno al metodo

Overloading di metodi

Domande...

- Perché due metodi con la stessa firma non possono stare nella stessa classe?
- Perché il nome dei parametri formali e il tipo del risultato del metodo non fanno parte della firma?

Firma del metodo m : prende n parametri di tipo T_i e restituisce un valore di tipo T

Formalmente l'invocazione di un metodo:

$$\frac{\forall i \in [1, n]. \Gamma \vdash_e e_i : T_i \quad obj : S \quad \boxed{T \ m \ (T_1 \ x_1, \dots, T_n \ x_n) \ C \in S}}{\Gamma \vdash_e obj.m(e_1, \dots, e_n) : T}$$

dove $T \ m(T_1 \ x_1, \dots, T_n \ x_n) \ C$ è la definizione del metodo nella classe:

- Ad es: `int somma(int x, int y) { return x+y; }`

Se il metodo non viene trovato?

Se il metodo non viene trovato si verifica un **errore in fase di compilazione**

- ▶ il type checker non può infatti applicare la precedente regola di inferenza per garantire che il metodo esista

Ma questo potrebbe anche dipendere dal fatto che il **controllo** è eseguito sul **tipo apparente** (o **statico**, quello dichiarato)

- ▶ Se sappiamo che nel tipo effettivo il metodo è presente, possiamo allora fare un **downcast esplicito** ↓
- ▶ Meglio se lo si fa dopo aver controllato il **tipo effettivo** (**tipo dinamico**) con il predicato **instanceof**
 - ▶ controllo di tipo dinamico basato sul **descrittore dell'oggetto**, che contiene il nome della classe usata per invocare la new

Type coercion (cast) di oggetti in Java

Downcast ↓ (da supertipo a sottotipo): serve un cast esplicito con controllo dinamico

```
Persona p = new Studente();  
if (p instanceof Studente) {  
    Studente s = (Studente) p;  
    int m = s.getMatricola();  
}
```

Studente <: Persona

Gli **upcast** ↑ (da sottotipo a supertipo) in Java sono invece **impliciti** (non è necessario scriverli)

► Poiché il polimorfismo per sottotipo (**subsumption**) si applica ovunque

NON è possibile effettuare cast tra tipi di oggetti che non hanno una relazione di sottotipo nella gerarchia delle classi o interfacce

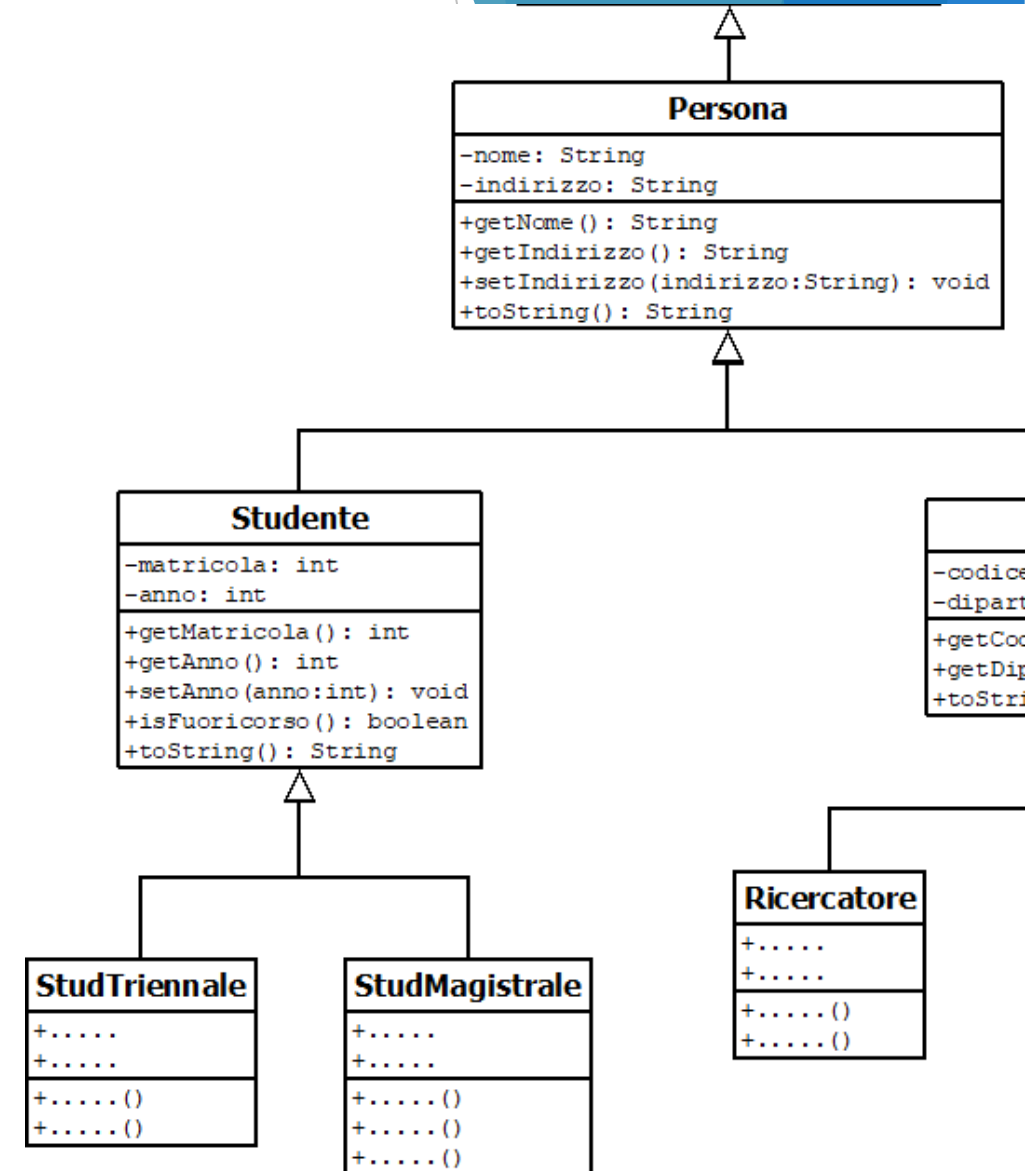
E a tempo di esecuzione?

Dynamic Dispatch dei metodi

Superati i controlli di tipo statici, a tempo di esecuzione la chiamata del metodo può trovarsi nei seguenti casi:

- ▶ il metodo **è presente nella classe** dell'oggetto
- ▶ il metodo **va cercato nelle super-classi** (le classi più in alto nella gerarchia)

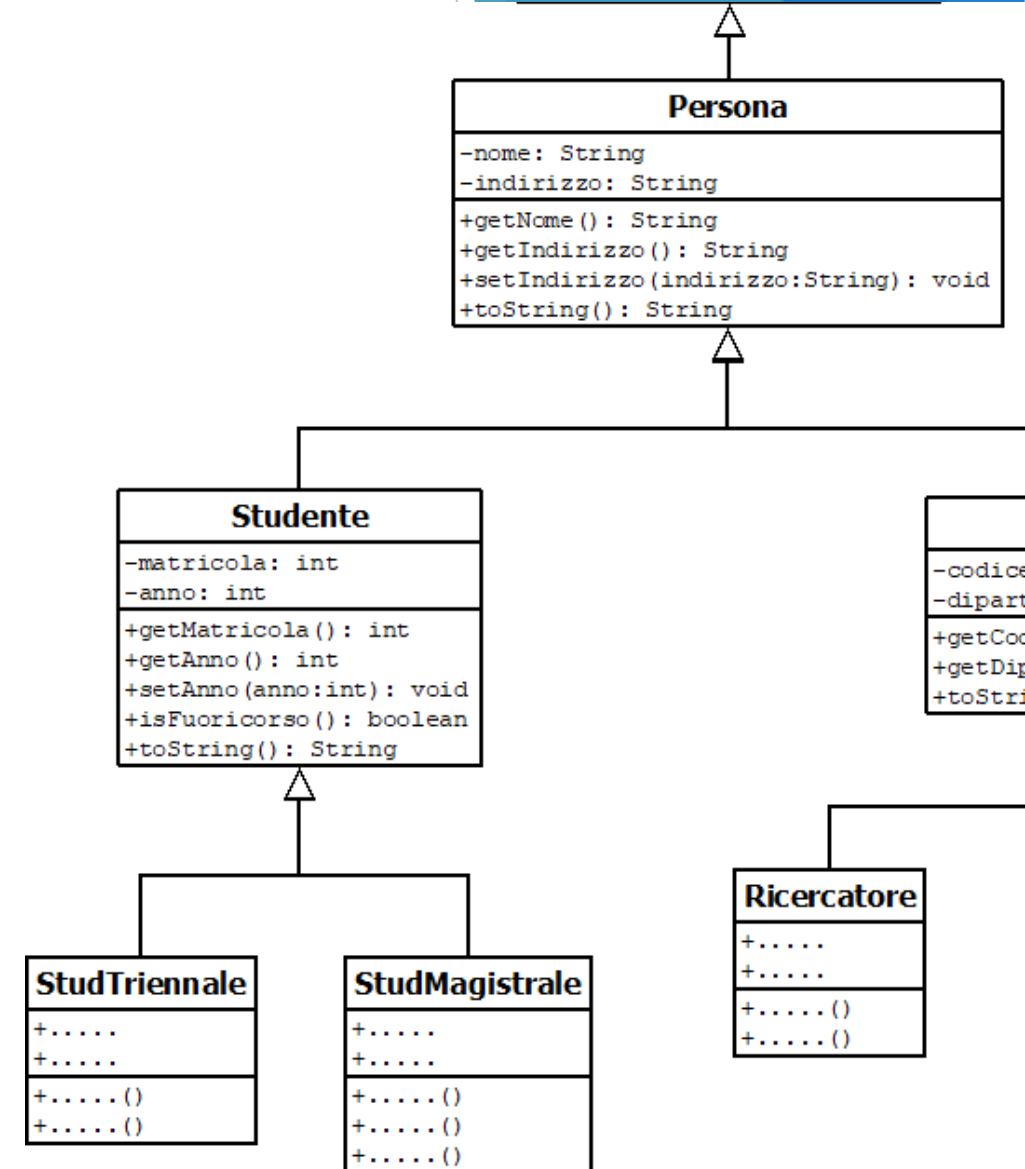
La JVM va alla ricerca del metodo partendo dalla classe corrente e risalendo la gerarchia (**DYNAMIC DISPATCH**, che vedremo poi)



La ricerca del metodo parte dalla classe che corrisponde al **TIPO EFFETTIVO** dell'oggetto

La JVM trova l'indicazione sul tipo effettivo nel **descrittore del dato** (oggetto) in memoria, inizializzato al momento della creazione dell'oggetto

stud: <class Studente>	
nome:	"Mario"
indirizzo:	"Via..."
matricola:	14421
anno:	2



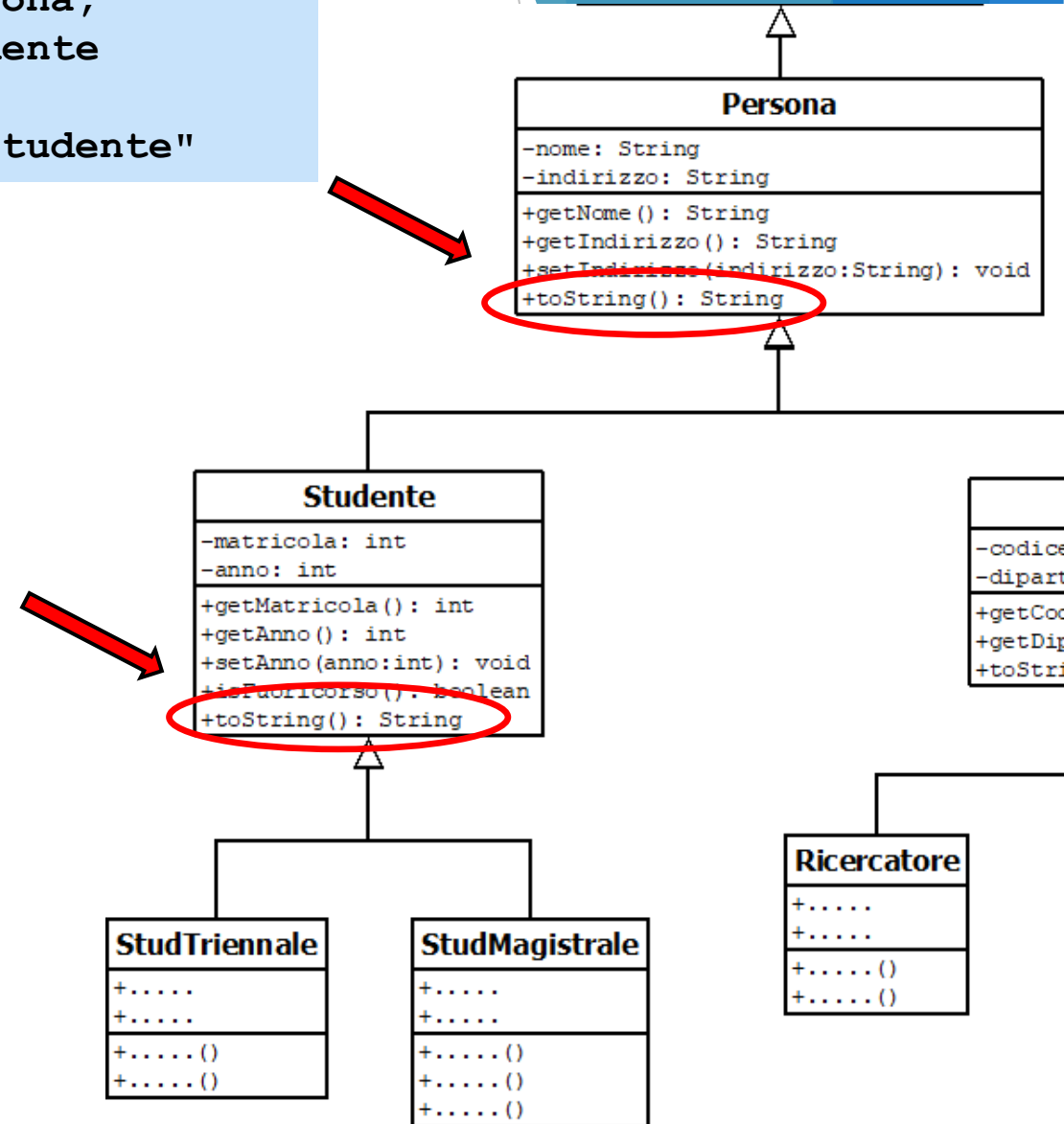
Overriding

```
Persona obj = new Studente ();  
// Tipo apparente: Persona,  
// Tipo effettivo: Studente  
obj.toString();  
// Output: "Metodo di Studente"
```

Risalendo la gerarchia è possibile incontrare
più metodi con la stessa firma

- a causa della **sovrascrittura (overriding)**
nelle sottoclassi dei metodi delle
superclassi

In questo caso, la JVM esegue il **primo
metodo che si trova risalendo la gerarchia**,
cioè la versione del metodo che è stata
ridefinita **più di recente** nella classe più
vicina al tipo effettivo dell'oggetto



Dynamic dispatch: realizzazione efficiente nella JVM

Visitare l'albero della gerarchia di classi **a ogni chiamata di metodo** è però una soluzione molto **inefficiente**

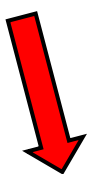
Per evitare questo problema, la JVM utilizza un approccio più ottimizzato basato su:

- ▶ **tabelle di metodi (dispatch vector)** [una per ogni classe]
(strutture dati con **puntatori** al codice dei metodi)
- ▶ **sharing strutturale**
(la tabella di una **sotto-classe** **riprende la struttura** della tabella della super-classe, aggiungendo righe per i nuovi metodi. I metodi ereditati dalla super-classe occupano le stesse posizioni nella tabella della sotto-classe)

Supporto al polimorfismo: quando un metodo è sovrascritto in una sottoclasse, la sua posizione nella tabella rimane la stessa, ma il puntatore viene aggiornato al nuovo codice

Nella JVM (a run time)

Descrittore dell'oggetto x



x: <class B>	
a1	10
a2	21
a3	57

CLASS A	
m1	
m2	

CLASS B	
m1	
m2	
m3	

```
class A {  
    int a1, a2;  
    void m1 ...;  
    void m2 ...;  
}
```

```
class B extends A  
    int a3;  
    void m1 ...;  
    void m3 ...;  
}
```



Tabella dei metodi della classe B
(dispatch vector)



Codice nell'ambiente delle classi

Sharing strutturale

La tabella della sotto-classe **riprende la struttura** (ordine delle righe) della tabella della superclasse

CLASS A	
m1	
m2	

CLASS B	
m1	
m2	
m3	

Esempio: il metodo **m2** è il **secondo** nella tabella sia di A che di tutte le sue sotto-classi (anche se magari i puntatori sono diversi a causa dell'overriding)

Dynamic dispatch: realizzazione efficiente nella JVM

Viene risolto **STATICAMENTE** il **dispatching dei metodi**, combinando informazioni determinabili in fase di compilazione con l'accesso ottimizzato a runtime

- ▶ Il **compilatore determina l'offset** del metodo nella tabella (la sua posizione nella tabella): l'**offset** è determinato sul **tipo apparente** (statico) dell'oggetto
- ▶ A tempo di esecuzione la JVM accede alla tabella dei metodi della classe del **tipo effettivo** (dinamico) usando quell'**offset**
- ▶ Anche se il **tipo effettivo** fosse **diverso** da quello apparente, sarà comunque una **sotto-classe** del **tipo apparente**. Grazie allo **sharing strutturale** la tabella del tipo effettivo conserva la struttura della tabella del tipo apparente, e l'offset calcolato in fase di compilazione rimane valido: la JVM accederà comunque al **metodo giusto**
- ▶ In caso di **overriding**, il **puntatore** nella tabella della sottoclasse verrà aggiornato **per puntare alla nuova versione del metodo**. Così, la JVM accede sempre alla versione corretta del metodo, anche se è stata sovrascritta

Dynamic dispatch: realizzazione efficiente nella JVM

 `A x = new B();`
`x.m2();`

```
class A {  
    int a1, a2;  
    void m1 ...;  
    void m2 ...;  
}
```

Index

 **0**

 **1**

Esempio:

- ▶ Il **tipo apparente** di x è A
- ▶ il compilatore trova il metodo m2 in seconda posizione nel codice della classe A (**indice 1**)
- ▶ il compilatore **traduce** la chiamata nel **bytecode** usando la seguente operazione (in bytecode)

`invokevirtual #1`  **Indice del metodo***

*non esattamente (piccola semplificazione):
#1 è in realtà l'indice del metodo in una
tabella di simboli da cui verrà poi derivato il
dispatch vector a run time

x: <class B>	
a1	10
a2	21
a3	57

x.m2 () ;

CLASS A	
m1	
m2	

CLASS B	
m1	
m2	
m3	

```
class A {
    int a1, a2;
    void m1 ...;
    void m2 ...;
}
```

```
class B extends A
    int a3;
    void m1 ...;
    void m3 ...;
}
```

- ▶ A tempo di **esecuzione** x avrà **tipo effettivo B**
- ▶ m2 in B ha la **stessa posizione** che ha in A (per lo **sharing strutturale**)
- ▶ **invokevirtual #1 chiama quindi il metodo giusto!**
- ▶ In caso di **overriding** (ad es. m1) avrebbe chiamato la **nuova versione** del metodo (seguendo il puntatore)

Dynamic dispatch: realizzazione efficiente nella JVM

Con il **dynamic dispatch** la risoluzione di un metodo avviene in **tempo costante**, perché la tabella consente

- ▶ un **accesso diretto**, tramite offset e puntatore, al codice del metodo, e questo
- ▶ **senza dover visitare** l'albero delle classi, senza cioè dover analizzare l'intera gerarchia

Accesso alle variabili d'istanza

Come fa un **metodo (non statico)** ad **accedere alle variabili d'istanza** dell'oggetto?

► Il compilatore aggiunge **this** come **parametro (implicito)** al metodo

```
public String getMatricola() { return matricola; }
```

e (**intuitivamente**, in realtà è un po' più complicato) lo **compila** in

```
public String getMatricola(Studente this) { return this.matricola; }
```

a **tempo di esecuzione** sarà passato il riferimento all'oggetto su cui il metodo è chiamato

Chiamata di metodi statici

Metodi statici: appartengono alla classe e non agli oggetti istanziati. Non possono accedere a variabili di istanza (non hanno il parametro implicito `this`)

A **run time** sono vengono trattati come procedure a top-level (funzioni globali)

- ▶ La loro esecuzione utilizza un **dispatch vector dedicato**, che consente di localizzarli rapidamente tramite l'operazione **invokestatic** nel bytecode
- ▶ Sostanzialmente non sono metodi: sono più simili a funzioni autonome associate alla classe